

Reconstruction algorithms for MA-TIRF 3D and 2D deconvolution

https://github.com/fplessec/matirf_benchmark_report

A reference for the six reconstruction algorithms in `matirf_python_plus` — Adam, PPXA, ADMM, PnP-HQS, ADMM-PnP and MCMC — on the MA-TIRF 3D depth-reconstruction problem and, as a well-posed companion, on 2D deconvolution.

Contents

1	The shared ground	1
2	Adam — general MAP minimiser	4
3	PPXA — proximal splitting twin of Adam	7
4	ADMM — sparsity by soft thresholding	7
5	PnP-HQS — a denoiser as the prior, on a schedule	10
6	ADMM-PnP — a denoiser as the prior, with a dual variable	12
7	MCMC — the MMSE estimator	14
8	Comparison — which algorithm, which denoiser, where	16
9	Decision guide	17
10	Reproducibility	17

1 The shared ground

Forward model. Both problems are linear with positivity:

$$g = Hf + n, \quad f \geq 0, \quad \max g = 1, \quad (1)$$

f the object, H the operator (for MA-TIRF, one $n_\theta \times n_z$ matrix per lateral column, mixing the n_z depth planes into the angle values; for deconvolution, a PSF convolution), n the additive noise, g peak-normalised.

Ill-posedness. The MA-TIRF per-column operator's singular values collapse:

	s_1	s_2	s_3	s_4	$s_5 \dots$
s	38.9	6.02	0.562	0.033	$< 10^{-3}$
s^2	1.5×10^3	36	0.32	1.1×10^{-3}	

so only s_1, s_2 (and partly s_3) are determined — ~ 47 of 50 directions lie in $\ker H$. The *prior*, not the optimiser, fixes them: MA-TIRF is **severely ill-posed**, with *depth collapse* (drift/flattening in z) as the failure mode. Deconvolution’s spectrum is continuous and smoothly decaying — far better conditioned — so ranges tied to the discrete s_i^2 do not transfer.

Scale-invariant metrics. Priors recover the object’s *amplitude* differently, so its absolute intensity is weakly determined while the axial *structure* is the goal. The depth metrics are therefore scale-invariant — an $\arg \max_z$ mislocalisation and a resolved/not count, unchanged under $f \mapsto cf$ ($c > 0$). Likewise a parameter in units of f is read against f ’s own scale, which is why the denoiser is fed $255 f/\text{peak}$, not $255 f$.

Objective. Adam and PPXA minimise

$$L(f) = (1 - \lambda) D(Hf, g) + \lambda r R(f), \quad f \geq 0, \lambda \in [0, 1], \quad (2)$$

with $D = \|Hf - g\|^2 / (2bn_g)$ (Gaussian noise variance b , n_g the pixel count of g) — a mean equal to $\frac{1}{2}$ at the truth, so $\text{chi2_ratio} = 2D$ — and $r = 1/R(f_{\text{init}})$ making λ a regularisation *share*. The others do not minimise L : **ADMM** minimises its own ℓ_1 energy; **PnP-HQS** and **ADMM-PnP** replace the prior by a denoiser (a fixed point, no explicit energy); **MCMC** returns the posterior *mean* by sampling.

Ridge start. Every solver starts from $f_0 = (H^\top H + \lambda_{rr} I)^{-1} H^\top g$ with $\lambda_{rr} = s_2^2$. A ridge weight keeps the directions $s_i^2 \gg \lambda_{rr}$ and damps $s_i^2 \ll \lambda_{rr}$, so s_2^2 retains the determined part at the right scale without amplifying noise — far better than the adjoint $H^\top g$ (orders of magnitude too large) or a zero start.

Metrics. Each isolates a different aspect of the reconstruction f against the truth f^* :

- **NMSE** $= \|f - f^*\|^2 / \|f^*\|^2$ — overall error, amplitude included (0 perfect, ~ 1 = no better than a blank image); the headline number.
- **PSNR** $= 10 \log_{10}(1/\text{MSE})$ [dB] — monotone in NMSE (identical ranking), in familiar, range-compressed units.
- **Depth error** [nm] = mean of $|\arg \max_z f - \arg \max_z f^*| \cdot dz$ over the signal columns — the *axial* mislocalisation, the MA-TIRF question. Scale-invariant; < 6 nm (one plane) is essentially exact, 50 nm to 100 nm signals depth collapse.
- **Stack recovery** $\in [0, 1]$ — the fraction of axial peak pairs at least 30 nm apart that are resolved as two: axial super-resolution, i.e. whether close planes are separated rather than merged.
- **SSIM** $\in [0, 1]$ (deconvolution only) — perceptual structural similarity, unreliable on the anisotropic 3D volume.
- **chi2_ratio** $= 2D = \text{mean residual}^2 / \text{noise variance}$ — ≈ 1 fits to the noise level, $\gg 1$ underfits, $\ll 1$ overfits. The only metric needing *no truth*, hence the sole quality number on the real esoubies data.

Benchmark. Every number comes from one campaign on three structures — **vesicles** (point-like), **fibres** (filaments), **cell** (membrane) — at noise $\{0, 0.02, 0.05\}$, plus 2D **img_001** and real **esoubies**; each run also records runtime, memory and status (an out-of-memory on a real stack disqualifies a solver). Each solver is run at a default and a best-found configuration (a denoiser sweep for the plug-and-play methods): the best prior and denoiser depend on the object.

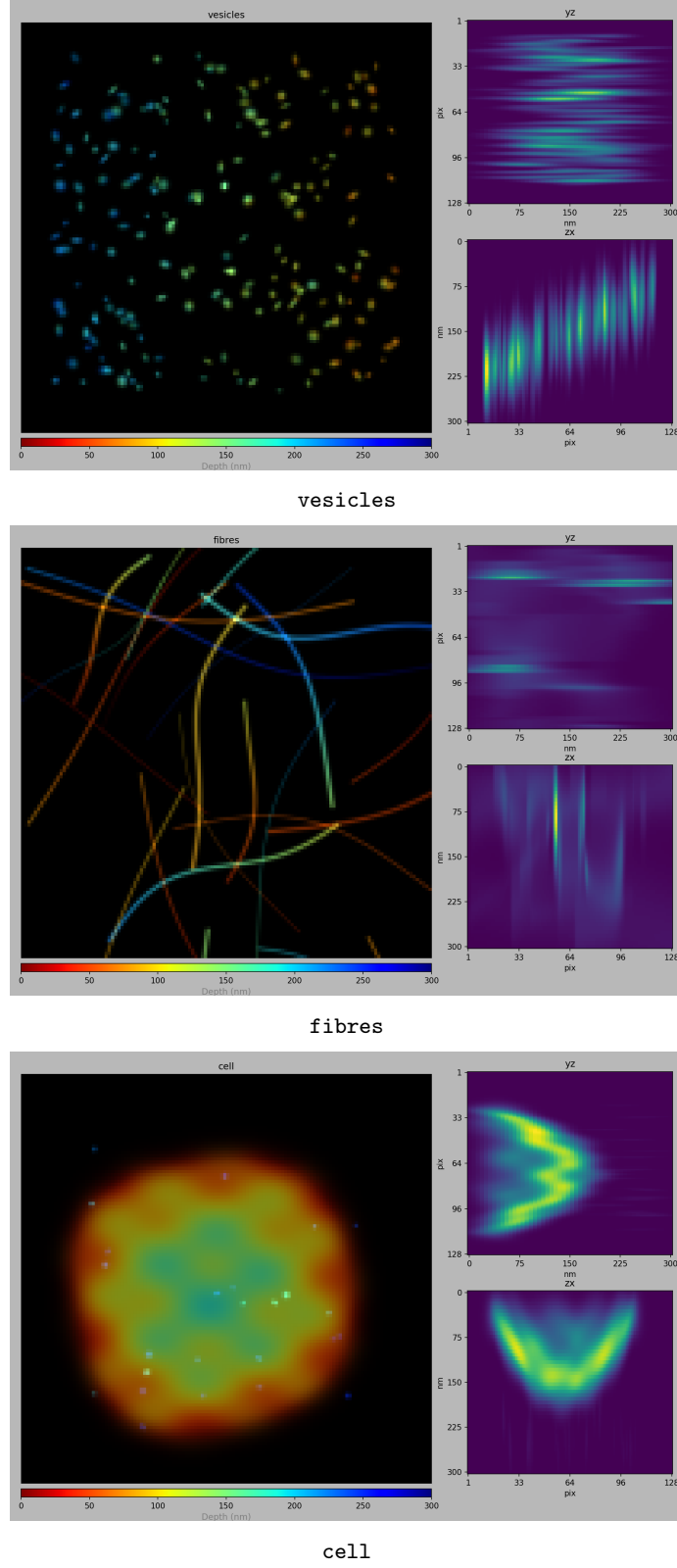


Figure 1: The three ground-truth structures — **vesicles** (point-like), **fibres** (filaments), **cell** (continuous membrane) — as depth map + yz/zx profiles.

2 Adam — general MAP minimiser

How it works. Adam minimises the objective (2) by gradient descent with a per-coordinate adaptive step, positivity by clamping, a learning rate halved on any loss increase, and a small-change step:

```

 $f \leftarrow f_0$  (start: the ridge  $s_2^2$  estimate)
repeat for each iteration:
   $g \leftarrow \nabla L(f)$  (gradient of the energy)
   $f \leftarrow f - \text{lr} \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$  (Adam step)
   $f \leftarrow \max(f, 0)$  (project onto  $f \geq 0$ )
every  $K$  steps: halve lr if the loss rose; stop if  $|\Delta L| < \text{EPS}$ 

```

where $\hat{m}, \hat{v}$ are Adam’s running estimates of the gradient and its square, lr the learning rate, ε a tiny stabiliser, K the check interval, EPS the plateau threshold. L is convex, so with a prior that weighs the minimiser is unique and Adam reaches it — the reconstruction is then only as good as the analytic prior R .

Key parameters. Only the prior **reg** and its share λ move the solution. We compare **L1** (sparsity) and **Tikhonov** (the ℓ_2 of the gradient, smoothness); Tikhonov is degree-2, so at the small scale of f it is $\sim 50\times$ weaker for a given λ and over-smooths. The rest are fixed convergence knobs: lr = 0.1, the ridge s_2^2 start, and a large budget with EPS = 10^{-8} so the plateau — not the budget — ends the run.

Results. L1 has a clear optimum around $\lambda \approx 0.2$; Tikhonov’s NMSE stays high and flat — it cannot localise a sparse object:

Table 1: Adam — prior shape at the reference structure (noise 0.02): NMSE and depth error versus lambda for the sparse (L1) and smooth (Tikhonov) priors.

prior	lambda	NMSE	depth err (nm)
l1	0.05	0.478	48
l1	0.1	0.507	54
tikhonov	0.05	0.405	66
tikhonov	0.2	0.459	80
tikhonov	0.3	0.483	87
tikhonov	0.5	0.527	97

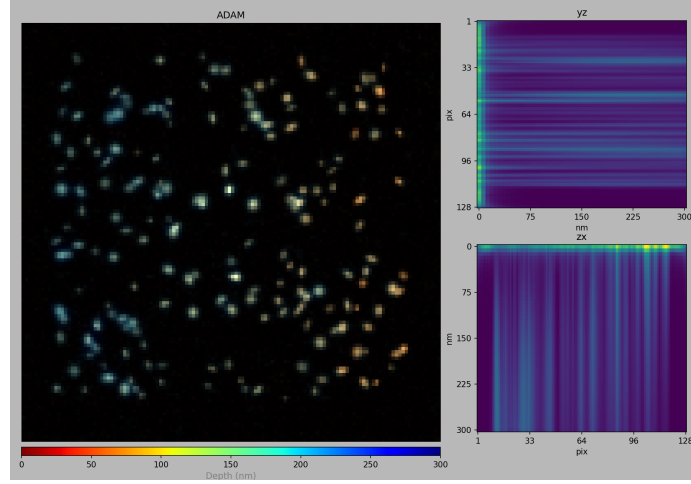
Table 2: ADAM on MA-TIRF — reconstruction metrics per ground truth and noise level.

dataset	noise	params	NMSE	PSNR	Depth Error (nm)	Stack Recovery	chi2_ratio
cell	0	reg=tikhonov, lambda=0.1	0.72	15.2	60	0	0.00507
cell	0	reg=none, lambda=0	0.476	17.0	21	0	1.17e-06
fibres	0	reg=tikhonov, lambda=0.1	0.321	29.0	58	0.241	0.0298
fibres	0	reg=none, lambda=0	0.0812	34.9	14	0	1.59e-05
vesicles	0	reg=tikhonov, lambda=0.1	0.481	29.3	69	0.5	0.048
vesicles	0	reg=none, lambda=0	0.0697	37.7	2	0	1.1e-05
cell	0.02	reg=tikhonov, lambda=0.1	0.534	16.5	26	0.0758	0.653
cell	0.02	reg=l1, lambda=0.2	0.277	19.4	16	0	0.591
fibres	0.02	reg=tikhonov, lambda=0.1	0.429	27.7	72	0.0431	0.433
fibres	0.02	reg=tikhonov, lambda=0.05	0.405	28.0	66	0.0862	0.415
vesicles	0.02	reg=tikhonov, lambda=0.1	0.552	28.7	69	0	0.423
cell	0.05	reg=tikhonov, lambda=0.1	0.584	16.1	30	0.0606	0.615
cell	0.05	reg=l1, lambda=0.2	0.487	16.9	22	0	0.596
fibres	0.05	reg=tikhonov, lambda=0.1	0.679	25.7	115	0.0431	0.411
vesicles	0.05	reg=tikhonov, lambda=0.1	0.787	27.1	146	0	0.391
esoubies	native	reg=l1, lambda=0.2	—	—	—	—	18.9

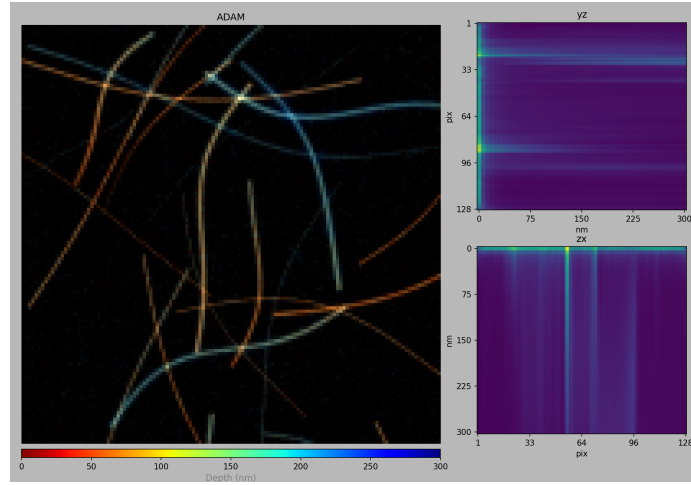
Table 3: ADAM on deconvolution — reconstruction metrics per noise level.

dataset	noise	params	NMSE	PSNR	SSIM	chi2_ratio
img_001	0	reg=tikhonov, lambda=0.1	0.195	14.1	0.302	0.021
img_001	0	reg=none, lambda=0	0.0191	24.4	0.694	1.96e-07
img_001	0.02	reg=tikhonov, lambda=0.1	0.0293	22.6	0.576	1.09
img_001	0.05	reg=tikhonov, lambda=0.1	0.0371	21.5	0.511	1.09

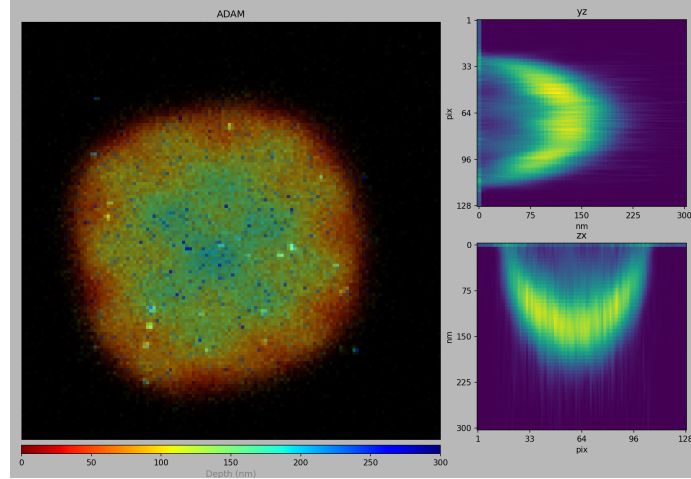
The prior winner tracks the object: L1 on the sparse structures (filaments, vesicles), the smooth prior closer to competitive on the continuous membrane.



vesicles — reg=tikhonov, lambda=0.1 (NMSE 0.552)



fibres — reg=tikhonov, lambda=0.05 (NMSE 0.405)



cell — reg=l1, lambda=0.2 (NMSE 0.277)

Figure 2: **ADAM** — best reconstruction on each structure at noise 0.02 (depth map + yz/zx profiles; the parameters under each panel reproduce it).

Limit. Adam has no failure of its own — it minimises the energy exactly — so on MA-TIRF the ceiling is entirely the analytic prior's, and none models a cell's depth structure as well as a learned denoiser. The only failures are at the ends of λ : $\lambda \rightarrow 0$ is noisy least squares, $\lambda \rightarrow 1$

drops the data.

3 PPXA — proximal splitting twin of Adam

How it works. PPXA minimises the *same* objective (2) as Adam, but by evaluating each term’s proximal operator in parallel, averaging, and over-relaxing:

$$\begin{aligned}
p_1 &\leftarrow (I + \gamma H^\top H)^{-1}(u_1 + \gamma H^\top g) && \text{(data prox: pull toward } g\text{)} \\
p_2 &\leftarrow \text{prox}_{\gamma \lambda R}(u_2) && \text{(prior prox)} \\
p_3 &\leftarrow \max(u_3, 0) && \text{(positivity prox)} \\
\bar{p} &\leftarrow \frac{1}{3}(p_1 + p_2 + p_3) && \text{(average)} \\
u_i &\leftarrow u_i + \rho_{\text{rel}}(2\bar{p} - f - p_i); \quad f \leftarrow f + \rho_{\text{rel}}(\bar{p} - f) && \text{(over-relax)}
\end{aligned}$$

where γ is the prox step (gain γs_i^2 per singular direction), u_1, u_2, u_3 the auxiliary variables, R the regulariser at share λ , and $\rho_{\text{rel}} \in (0, 2)$ the relaxation. Minimising the same convex L , it reaches the *same image* as Adam.

Key parameters. The same solution axes as Adam (`reg`, λ); its own knobs set speed only, provided the proxies are exact: $\gamma \in [1/s_1^2, 1/s_2^2]$ (out of range it can stall) and $\rho_{\text{rel}} = 1.5$.

Results. Run identically to Adam. The campaign confirms the framework claim — *Adam and PPXA return the same image* on the shared L1 objective (reference structure, noise 0.02):

Table 4: PPXA on MA-TIRF — reconstruction metrics per ground truth and noise level.

dataset	noise	params	NMSE	PSNR	Depth Error (nm)	Stack Recovery	chi2_ratio
fibres	0.02	reg=l1, lambda=0.2	0.855	24.7	73	0	0.446
esoubies	native	reg=l1, lambda=0.2	—	—	—	—	18.8

Limit. PPXA shares Adam’s ceiling (only as good as the analytic prior) and adds two costs: it needs *proximable* priors (Adam takes any differentiable one), and its splitting is markedly slower here (median runtime several times Adam’s).

4 ADMM — sparsity by soft thresholding

How it works. ADMM is the MAP estimator for a *sparse*, non-negative object: its prior is hard-wired ($\ell_1 + \text{positivity}$), so it ignores R, λ, r . It alternates a data step, a soft threshold and a dual update:

$$\begin{aligned}
t &\leftarrow \kappa \max(g)/s_1 && \text{(soft-threshold level)} \\
\textbf{repeat} &\text{ for each iteration:} \\
u &\leftarrow (H^\top H + \mu I)^{-1}(H^\top g + \mu(f - \eta)) && \text{(data step: ridge fit pulled to } f - \eta\text{)} \\
f &\leftarrow \max(u + \eta - t, 0) && \text{(soft-threshold at } t, \text{ then } f \geq 0\text{)} \\
\eta &\leftarrow \eta + \mu(u - f) && \text{(dual update)}
\end{aligned}$$

where κ (`threshold_ratio`) sets the threshold t , η is the dual variable, and μ the penalty. The Laplacian prior $\exp(-\tau \|f\|_1)$ suits point-like and filamentary objects, not a continuous membrane.

Key parameters. κ sets the sparsity but its useful band cannot be predicted a priori (the data step spreads the object across the null space, so a diffuse truth can be emptied by a modest κ); sweep it log-spaced and raise it with the noise. μ carries three coupled roles at once (data-step ridge, prior weight $\tau = \mu t$, dual step) and is capped: $\mu > 1.618$ diverges. `iter`= 200.

Results. On clean data the soft threshold localises a concentrated object almost exactly:

Table 5: ADMM on MA-TIRF — reconstruction metrics per ground truth and noise level.

dataset	noise	params	NMSE	PSNR	Depth Error (nm)	Stack Recovery	chi2_ratio
cell	0	kappa=0.1, mu=0.5	0.0318	28.8	4	0.636	5.2e-08
fibres	0	kappa=0.1, mu=0.5	0.251	30.1	15	0.336	0.000278
fibres	0	kappa=0.01, mu=0.5	0.0406	38.0	10	0.966	4.86e-06
vesicles	0	kappa=0.1, mu=0.5	0.142	34.6	8	0	0.00024
vesicles	0	kappa=0.01, mu=0.5	0.021	42.9	8	1	5.3e-06
cell	0.02	kappa=0.1, mu=0.5	0.619	15.9	41	0.152	0.604
fibres	0.02	kappa=0.1, mu=0.5	0.882	24.6	64	0.0517	0.436
vesicles	0.02	kappa=0.1, mu=0.5	0.848	26.8	66	0	0.413
cell	0.05	kappa=0.1, mu=0.5	0.899	14.3	68	0.0758	0.589
fibres	0.05	kappa=0.1, mu=0.5	0.973	24.2	95	0.0172	0.411
vesicles	0.05	kappa=0.1, mu=0.5	0.974	26.2	94	0	0.39
esoubies	native	kappa=0.1, mu=0.5	—	—	—	—	19.2

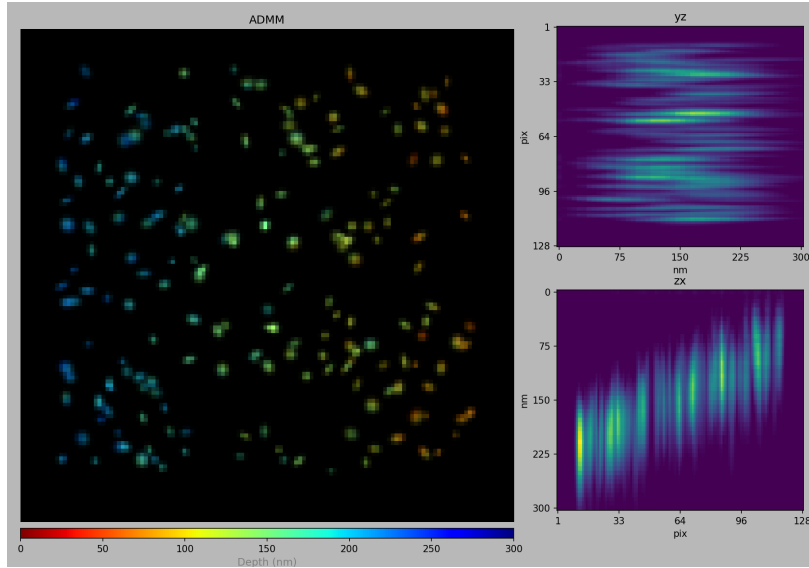
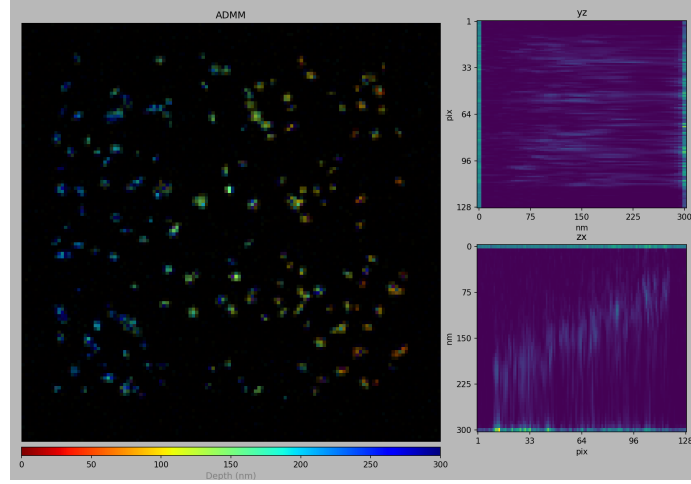
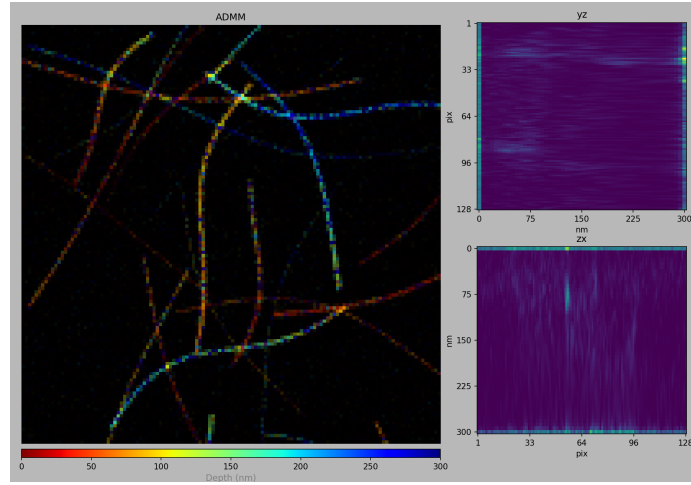


Figure 3: ADMM on **vesicles** at zero noise — the sparsity prior localises the point-like structure well.

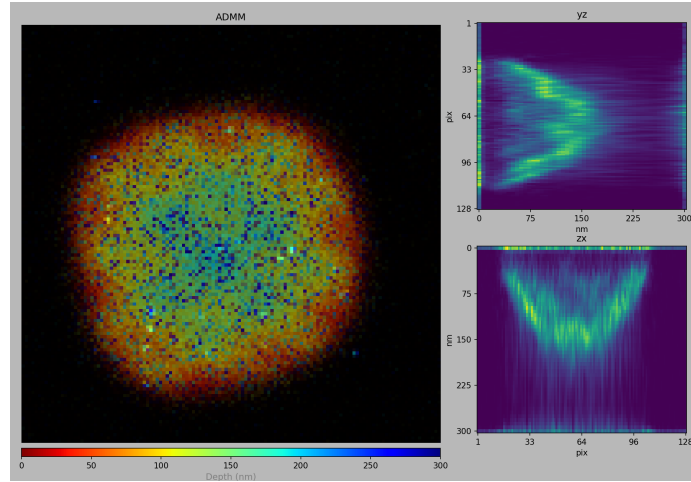
ADMM is in fact the best solver at *zero noise* on all three structures; under noise it degrades sharply (the threshold empties the spread-out, noisy estimate) — visible in the gallery below at noise 0.02:



vesicles — $\kappa=0.1$, $\mu=0.5$ (NMSE 0.848)



fibres — $\kappa=0.1$, $\mu=0.5$ (NMSE 0.882)



cell — $\kappa=0.1$, $\mu=0.5$ (NMSE 0.619)

Figure 4: **ADMM** — best reconstruction on each structure at noise 0.02 (depth map + yz/zx profiles; the parameters under each panel reproduce it).

Limit. The wall is the prior. On a diffuse or continuous object ℓ_1 is the wrong model, so there is little-to-no useful κ band — κ too large empties the image (the dominant failure), $\kappa \approx 0$ is noisy. Use ADMM for sparse objects at low noise; do not expect it to reconstruct a membrane.

5 PnP-HQS — a denoiser as the prior, on a schedule

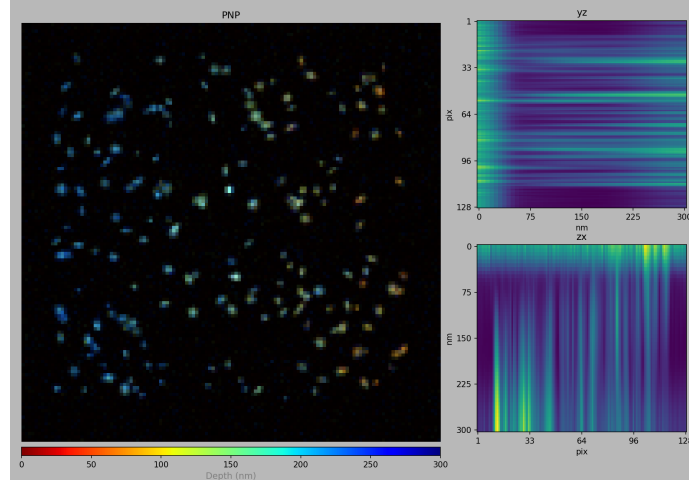
How it works. Plug-and-Play half-quadratic splitting replaces the prior’s proximal operator by a *denoiser* D_σ , over a short coarse-to-fine annealing schedule (not to convergence):

$$\begin{aligned}
 z &\leftarrow f_0 && (\text{start: the ridge } s_2^2 \text{ estimate}) \\
 \text{for } k = 1 \dots \text{iter}, \text{ with } \sigma_k \downarrow \text{ and } \alpha_k \uparrow: &&& \\
 f &\leftarrow (H^\top H + \alpha_k I)^{-1} (H^\top g + \alpha_k z) && (\text{data step: fit } g, \text{ pulled toward } z) \\
 z &\leftarrow \frac{\text{peak}}{255} D_{\sigma_k}(255 f / \text{peak}) && (\text{denoise at level } \sigma_k, \text{ scale-free})
 \end{aligned}$$

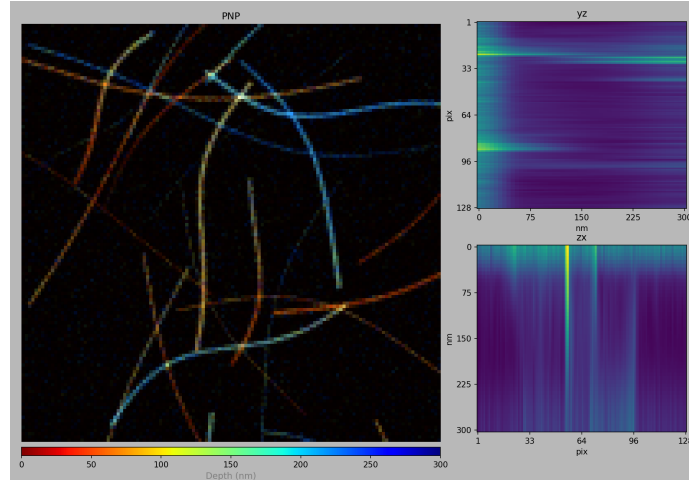
where D_σ is the denoiser, σ_k its level (annealed from σ down to the measurement’s noise), α_k the data-coupling weight (annealed *up* to $\lambda_{kz} = \alpha_{\text{final}}$), and peak the start’s peak. Since $s_i^2 \ll \alpha$ keeps the denoised z , the ~ 47 null-space directions are the denoiser’s — it has no dual variable, so the schedule alone drives the prior in.

Key parameters. The denoiser and its level σ ; the final data weight $\lambda_{kz} = \alpha_{\text{final}}$ is a **threshold on the eigenvalues** s_i^2 : at $\lambda_{kz} \approx 12$ only s_1, s_2 follow the data and the whole null space goes to the prior — what an ill-posed problem needs. (The DPIR default $\lambda_{kz} = 0.23$, tuned for deconvolution, under-uses the prior and is data-starved here.) **iter** is a real lever too — more schedule steps keep improving, past the usual [8, 30].

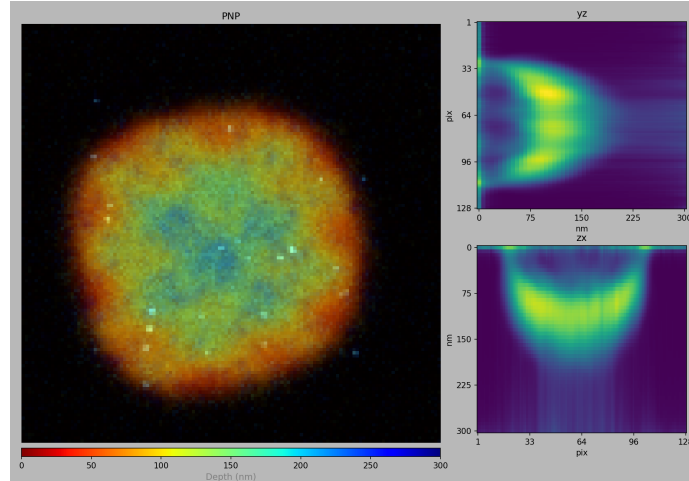
Results. With $\lambda_{kz} \approx 12$, **iter** ≈ 40 and the right denoiser, PnP reconstructs well; the best denoiser depends on the object (Table 8) — Gaussian on the filament network, TV Bregman on the membrane under noise, DCT competitive.



vesicles — denoiser=Gaussian, sigma=25, lambda_kz=12, iter=40 (NMSE 0.522)



fibres — denoiser=Gaussian, sigma=25, lambda_kz=12, iter=40 (NMSE 0.385)



cell — denoiser=TV Bregman, sigma=40, lambda_kz=12, iter=40 (NMSE 0.262)

Figure 5: **PNP** — best reconstruction on each structure at noise 0.02 (depth map + yz/zx profiles; the parameters under each panel reproduce it).

Limit. Two intrinsic limits. *Scale*: a denoiser transfers only if fed at the scale it was built for — the 255 f/peak bridge is what makes even DCT usable on MA-TIRF (it is *not* categorically broken, contrary to the naive-scale belief). *Cost*: a 3D denoise on the full volume every schedule step makes it memory- and time-hungry on a real stack (Sec. 8). Being dual-free, PnP needs a high λ_{kz} to force the denoiser into the null space.

6 ADMM-PnP — a denoiser as the prior, with a dual variable

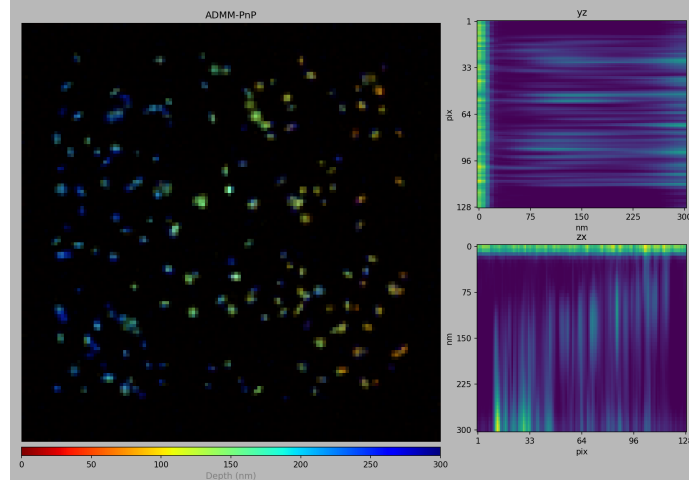
How it works. ADMM-PnP is the augmented-Lagrangian counterpart of PnP: the same split, but a **dual variable** u instead of a schedule, so ρ, σ stay fixed and it runs to a fixed point:

$$\begin{array}{ll}
x \leftarrow f_0; & v \leftarrow x; \quad u \leftarrow 0 \\
\text{repeat for each iteration:} & \text{(start; } u \text{ is the dual variable)} \\
x \leftarrow (H^\top H + \rho I)^{-1}(H^\top g + \rho(v - u)) & \text{(data step: fit } g, \text{ pulled to } v - u) \\
v \leftarrow \max\left(\frac{\text{peak}}{255} D_\sigma(255(x + u)/\text{peak}), 0\right) & \text{(denoise at fixed } \sigma, \text{ then } v \geq 0) \\
u \leftarrow u + (x - v) & \text{(dual update)}
\end{array}$$

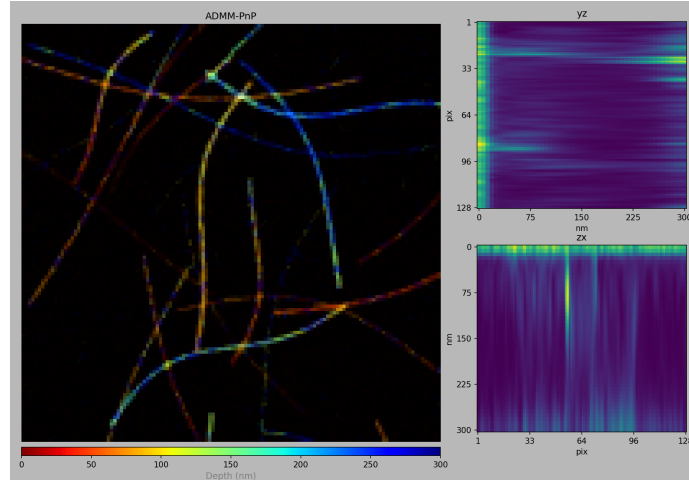
where D_σ is the denoiser at fixed level σ , ρ the penalty, x the data copy, v the denoised copy, u the dual variable that accumulates the mismatch. The dual, not a schedule, fills the null space.

Key parameters. The denoiser, σ , and ρ . The prior’s strength is the product $\rho\sigma^2$ (confirmed: at fixed $\rho\sigma^2$ the result is unchanged; the sweet spot is $\rho\sigma^2 \approx 70$). Crucially ρ is *not* the eigenvalue threshold that λ_{kz} is in PnP — the dual does the null-space filling, so a *low* ρ (moderate prior) wins and a large ρ over-smooths. The opposite of PnP.

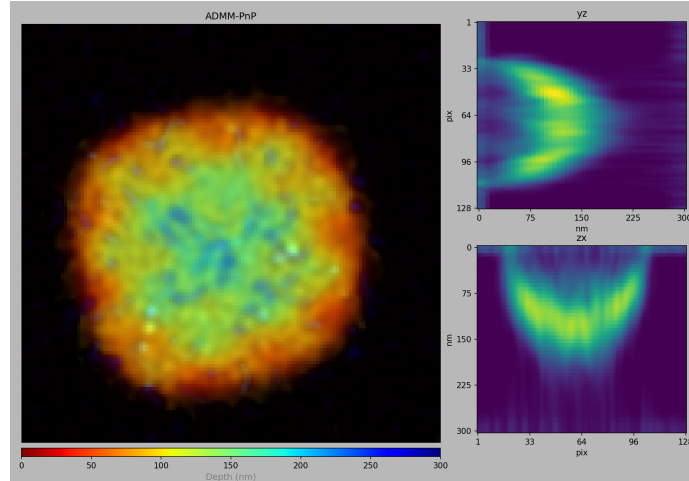
Results. Typically the strongest reconstructor under noise. The best denoiser depends on the object (Table 8), with one firm rule: **TV Bregman collapses in the dual iteration** (its data fit blows up) on every structure — Gaussian wins on the membrane, DCT on the sparse structures under noise.



vesicles — denoiser=Gaussian, sigma=25, rho=0.1 (NMSE 0.546)



fibres — denoiser=Gaussian, sigma=25, rho=0.1 (NMSE 0.509)



cell — denoiser=Gaussian, sigma=50, rho=0.03 (NMSE 0.177)

Figure 6: **ADMM-PnP** — best reconstruction on each structure at noise 0.02 (depth map + yz/zx profiles; the parameters under each panel reproduce it).

Limit. The family's limits: the denoiser must be fed at the right scale ($\div$ peak), and a 3D denoise on the full volume every iteration is the memory bind — ADMM-PnP peaked near 10 GB

and ran out of memory on the real **esoubies** stack. It needs a tiling/precision budget before running on full real volumes.

7 MCMC — the MMSE estimator

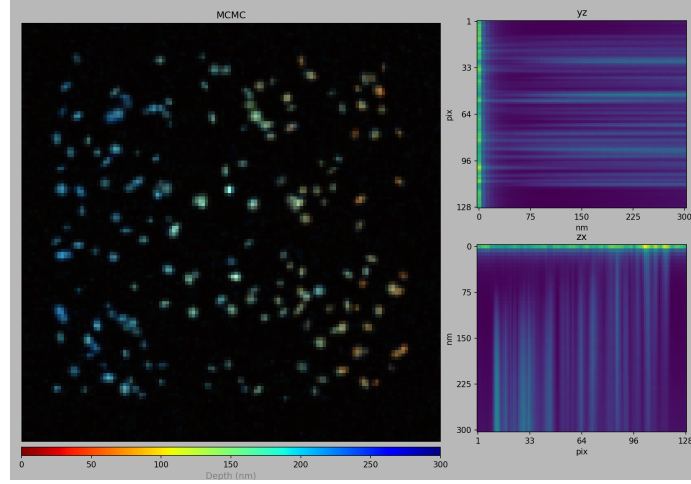
How it works. MCMC is the only *MMSE* method: it returns the posterior mean, not a mode, by a Metropolis-Hastings chain averaged after a burn-in:

$f \leftarrow f_0$	<i>(start: the ridge s_2^2 estimate)</i>
repeat for each sample:	
$z \leftarrow f + (H^\top H + \lambda_{rr} I)^{-1} H^\top (g - H f)$	<i>(pull toward the data)</i>
$z \leftarrow z + \sigma \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$	<i>(explore)</i>
$z \leftarrow D_\sigma(\max(z, 0))$	<i>(positivity, then denoise)</i>
accept z with probability $\min(1, e^{-\Delta D/\beta})$	<i>(Metropolis test on D)</i>
return the mean of the states after burn-in	<i>(the MMSE estimate)</i>

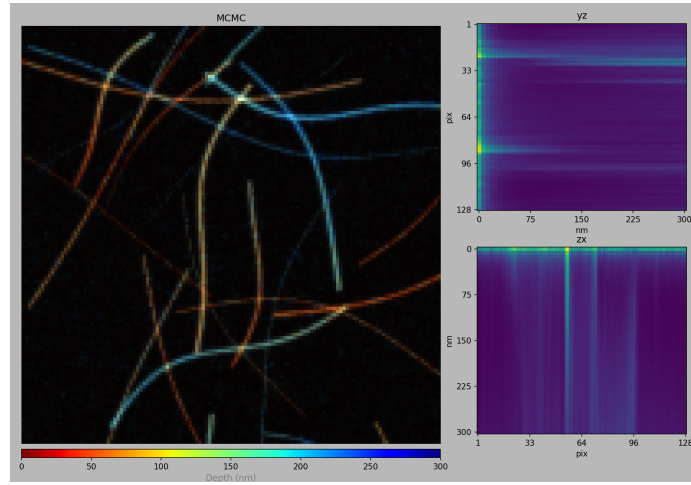
where λ_{rr} is the data-step ridge weight, σ the exploration/denoising level (a fraction of the peak), ε Gaussian noise, D_σ the denoiser, β the acceptance temperature, and ΔD the change in the data term. It is dual-free, like PnP.

Key parameters. The denoiser and σ ; but the decisive knob is λ_{rr} , whose useful window is $[s_3^2, s_2^2] \approx [0.3, 36]$. $\lambda_{rr} = s_1 = 38.9$ sits *above* it, so the data step barely pulls and the chain drifts; bring $\lambda_{rr} \approx 5$ into the window and take enough samples — this is the single largest lever.

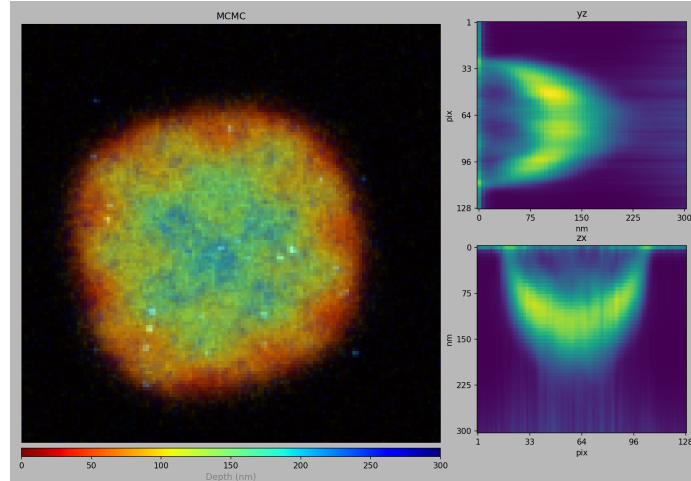
Results. With $\lambda_{rr} \approx 5$, MCMC **beats the ridge start on MA-TIRF** and *excels* on the well-posed deconvolution. The best denoiser is object-dependent (Table 8): Gaussian at low noise, shifting to TV Bregman (membrane) or DCT (sparse) as noise rises.



vesicles — denoiser=TV Bregman, sigma=0.02,
lambda_rr=38.863, iter=300 (NMSE 0.512)



fibres — denoiser=TV Bregman, sigma=0.02,
lambda_rr=38.863, iter=300 (NMSE 0.429)



cell — denoiser=TV Bregman, sigma=0.01, lambda_rr=5,
iter=300 (NMSE 0.235)

Figure 7: **MCMC** — best reconstruction on each structure at noise 0.02 (depth map + yz/zx profiles; the parameters under each panel reproduce it).

Limit. With a good λ_{rr} MCMC is competitive with the MAP methods, though usually behind the plug-and-play family. Its honest limits: the denoiser must be 3D-appropriate, and **None** (no prior) cannot beat its own start.

8 Comparison — which algorithm, which denoiser, where

Which solver wins. For every (structure, noise) the solver with the best NMSE and its curated metrics:

Table 6: Best algorithm by NMSE on MA-TIRF — winner per structure and noise level.

dataset	noise	best (by NMSE)	params	NMSE	PSNR	Depth Error (nm)	Stack Recovery	chi2_ratio
cell	0	ADMM	kappa=0.1, mu=0.5	0.0318	28.8	4	0.636	5.2e-08
cell	0.02	ADMM-PnP	denoiser=Gaussian, sigma=50, rho=0.03	0.177	21.3	9	0	0.96
cell	0.05	ADMM-PnP	denoiser=Gaussian, sigma=50, rho=0.03	0.307	18.9	21	0	0.903
fibres	0	ADMM	kappa=0.01, mu=0.5	0.0406	38.0	10	0.966	4.86e-06
fibres	0.02	PNP	denoiser=Gaussian, sigma=25, lambda_kz=12, iter=40	0.385	28.2	59	0.0172	0.527
fibres	0.05	PNP	denoiser=Gaussian, sigma=25, lambda_kz=12, iter=40	0.653	25.9	85	0.069	0.459
vesicles	0	ADMM	kappa=0.01, mu=0.5	0.021	42.9	8	1	5.3e-06
vesicles	0.02	MCMC	denoiser=TV Bregman, sigma=0.02, lambda_rr=38.863, iter=300	0.512	29.0	61	1	0.609
vesicles	0.05	PNP	denoiser=Gaussian, sigma=25, lambda_kz=12, iter=40	0.732	27.5	98	0	0.435

Table 7: Best algorithm by NMSE on deconvolution — winner per structure and noise level.

dataset	noise	best (by NMSE)	params	NMSE	PSNR	SSIM	chi2_ratio
img_001	0	ADAM	reg=none, lambda=0	0.0191	24.4	0.694	1.96e-07
img_001	0.02	MCMC	denoiser=TV Bregman, sigma=0.03	0.0245	23.4	0.596	0.98
img_001	0.05	ADAM	reg=tikhonov, lambda=0.1	0.0371	21.5	0.511	1.09

On clean data ADMM (sparsity) wins on all three structures; as noise rises the plug-and-play family and MCMC take over.

Which denoiser. There is **no universal best denoiser** — it depends on the object, the noise, and the algorithm:

Table 8: Winning denoiser (best NMSE) per structure, noise, and plug-and-play algorithm.

structure	noise	PnP	ADMM-PnP	MCMC
cell	0	DCT (0.191)	Gaussian (0.157)	Gaussian (0.227)
fibres	0	Gaussian (0.188)	DCT (0.23)	Gaussian (0.115)
vesicles	0	TV Bregman (0.303)	Gaussian (0.311)	Gaussian (0.166)
cell	0.02	TV Bregman (0.262)	Gaussian (0.177)	TV Bregman (0.235)
fibres	0.02	Gaussian (0.385)	DCT (0.568)	DCT (0.511)
vesicles	0.02	Gaussian (0.522)	DCT (0.603)	DCT (0.588)
cell	0.05	TV Bregman (0.437)	Gaussian (0.307)	TV Bregman (0.453)
fibres	0.05	Gaussian (0.653)	TV Bregman (0.862)	DCT (0.856)
vesicles	0.05	Gaussian (0.732)	TV Bregman (0.87)	DCT (0.889)

Two facts stand out. *One firm algorithm rule:* TV Bregman collapses inside the dual-variable ADMM-PnP on every structure, while it is competitive-to-best in the dual-free PnP and MCMC — the dual, more than a single “best denoiser”, flips the ranking. *DCT is rehabilitated:* long thought unusable on MA-TIRF, it is often second, sometimes first, once fed at the right scale ($\div$ peak). Bilateral and Wiener, consistently worst, were dropped.

Cost and robustness. Runtime, peak memory and rejection rate per solver — including the PnP family’s memory pressure on the full **esoubies** volume (every solver is run on it):

Table 9: Cost and robustness per solver — runs, rejection rate, failures, median runtime, peak memory and top findings.

solver	runs	rejected	failed/timeout	median s	max mem (MB)	top findings
ADAM	45	4 (8%)	0	17.2	2176	no-better×72, depth-shift×18, collapsed×4
ADMM	19	1 (5%)	0	1.7	2204	no-better×20, depth-shift×10, collapsed×1
ADMM-PnP	37	3 (8%)	1	14.7	10210	no-better×61, depth-shift×18, collapsed×2
MCMC	39	0 (0%)	0	9.7	2236	no-better×47, depth-shift×17
PnP	37	0 (0%)	0	3.7	3591	no-better×40, depth-shift×18
PPXA	2	0 (0%)	0	291.7	1656	no-better×3, depth-shift×1

The pattern. No single algorithm wins everywhere: the report’s thesis is a map, not a champion. Match the prior to the structure and the noise, and match the denoiser to *both* the object and the algorithm’s mechanism (dual or not).

9 Decision guide

The actionable one page.

your data	use	starting parameters
sparse object, low noise	ADMM	$\kappa \in [0.01, 0.3]$ (raise with noise), $\mu \in [0.5, 1]$
continuous object, noisy	ADMM-PnP	Gaussian, $\sigma \in [25, 50]$ (raise with noise), $\rho \approx 0.1$
most versatile / unsure	Adam	L1, $\lambda \approx 0.2$ (Tikhonov if you want a smooth prior; $\lambda = 0$ only on clean data)
uncertainty / MMSE, or a middle-ground reconstructor	MCMC	TV Bregman, $\sigma \approx 0.01$, $\lambda_{rr} \approx 5$ (not the default s_1); beats the ridge on MA-TIRF, excels on deconvolution

Rules of thumb. Noise demands a *stronger* prior — the best regularisation grows with the noise level. Match the prior to the structure: sparsity (Adam + L1, or ADMM) for point-like objects, a plug-and-play denoiser (ADMM-PnP) for continuous ones; a smooth analytic prior (Tikhonov) is a safe but blunt default. There is no universal best denoiser — match it to the object and noise (Gaussian at low noise; TV Bregman for a membrane, DCT for sparse structures under noise), with one firm rule: **never TV Bregman inside ADMM-PnP** (it collapses). Budget memory before running the PnP family on full real volumes (ADMM-PnP peaked at ~10 GB). Every parameter in units of f is on a ~0.02 scale, not 1 — including a denoiser σ , which is why the framework feeds the denoiser 255 f /peak.

10 Reproducibility

Every number and figure is regenerable from **matirf_python_plus** (tag v2.0) by one curated campaign.

Run the campaign.

```
python -m benchmarks run          # start (or resume) the curated campaign
python -m benchmarks status       # how far it got, by status
python -m benchmarks stop        # stop cleanly before the next run (resumable)
python -m benchmarks specs -v    # the exact run list, without running anything
```

Add `--timeout <s>` to cap each run. Every run is an isolated process with a per-run timeout; a crash or out-of-memory is contained and the campaign resumes where it stopped. Each config is written in full (`benchmarks/campaign.py`): every parameter a solver reads, including the ones held fixed for the whole campaign, is explicit — so a run reproduces from its `config.toml` alone. Results land in `benchmarks/results/main/` (git-ignored: large, regenerated).

Build this report.

```
python -m benchmarks analyze      # writes tables/ + figures/
latexmk -pdf main.tex
```

`analyze.py` reads the durable `journal.jsonl` and writes: one standard-vs-optimal table per algorithm (`tables/algo_*.tex`), the Adam λ -shape (`tables/adam_shape.tex`), the cross-comparison and cost tables, and the depth-map/profile PNGs of each showcase reconstruction into `figures/`. The reconstructions themselves are not committed (large, regenerable from the config).